A Privacy-Preserving Retrieval-Augmented Incident Assistant: Unified Architecture, Hybrid Retrieval, Robustness, and Chat-to-Report Automation

Ayoub Aarab
Master Data Science — End-of-Year Internship Project
Ecole Normale Supérieure, Martil

July 16, 2026

Abstract

This report presents the complete design, optimization, and evaluation of a unified incident-management platform that merges two previously separate applications — a React-based incident-report generator and a Streamlit prototype chatbot — into a single, fully local, production-oriented system. The platform runs entirely on self-hosted models (Ollama for text generation, a local sentence-transformer for embeddings) to satisfy a strict data-privacy constraint: no incident content ever leaves the organization's infrastructure.

The engineering contribution is a maintainable modular monolith (FastAPI backend, React frontend, Docker deployment) whose two modules share a single report schema, making the system's two halves genuinely interoperable. The data-science contribution is a **hybrid retrieval** component that fuses a dense embedding search with a sparse BM25 lexical index via Reciprocal Rank Fusion (RRF), evaluated on a labeled benchmark of 272 queries over a corpus of 68 incident reports spanning nine IT domains.

Our evaluation demonstrates that: (1) **hybrid retrieval** reaches Recall@5 = 1.000 and MRR = 0.987, versus 0.768 and 0.678 for dense-only search; (2) the improvement is concentrated on **exact-identifier queries**, where dense-only Recall@5 collapses to 0.221 while hybrid recovers 1.000; (3) retrieval quality is **insensitive to chunk size** and plateaus at $k \geq 3$; and (4) the residual failure rate is **0.0%** (0 of 272 queries). On top of this core, we contribute a robustness layer (intent-based abstention, prompt-injection defense, grounded generation), a human-feedback loop, and a **chat-to-report** feature that converts a diagnosed conversation into a schema-validated report with no invented fields. The final system comprises 27 backend modules and is covered by 115 automated tests.

Contents

1 Introduction and Context

1.1 Industrial Context

Incident management in large IT and telecom operations rests on two complementary activities. The first is *documentation*: capturing each resolved incident as a structured report (symptom, root cause, resolution steps, supporting code or configuration) so that institutional knowledge is preserved. The second is *retrieval and reuse*: when a new incident arrives, quickly finding similar past incidents to accelerate diagnosis and resolution.

In the host organization, these two activities were served by two disconnected tools:

- An **incident-report generator**: a modern React/TypeScript single-page application with a rich block-based editor (headings, paragraphs, lists, code snippets, tables, images), backed by a small Express server that persisted reports as JSON files.
- A **chatbot prototype**: a Python application built on Streamlit that performed retrieval over the generated reports and used a local language model to propose resolutions. The Streamlit interface was explicitly a throwaway prototype.

These tools lived on separate Git branches, were written in different languages, and shared no interface. The internship mandate was to merge them into a single production-oriented application with one coherent user experience.

1.2 Problem Statement and Constraints

The project was shaped by four hard constraints, each of which materially affected the design:

1. **Privacy (non-negotiable).** Incident data frequently contains sensitive operational detail. The organization required that no incident content ever be sent to a third-party cloud API. Consequently, all inference — both embeddings and text generation — had to run on self-hosted, local models. This single constraint ruled out the simplest high-quality path (a hosted LLM API) and set the performance ceiling for the entire system.
2. **Accuracy.** Retrieval had to surface genuinely relevant historical incidents. In particular it had to work on *exact identifiers* — incident IDs, error codes, table and function names — where purely semantic (embedding) search is known to be weak.
3. **Robustness.** As an LLM-facing tool used under operational pressure, the assistant had to behave safely on ambiguous, out-of-scope, or adversarial input rather than confidently fabricating a root cause.
4. **Maintainability.** The result had to be a tested, deployable, single-command application — not a research notebook.

1.3 Contributions

This report documents the full journey and makes the following contributions:

- A **unified architecture** (modular monolith) that absorbs the existing React report generator and re-implements the chatbot as a headless service behind one API.
- A **hybrid dense/sparse retriever** fused with RRF, and a rigorous **evaluation** on a labeled benchmark with standard IR metrics and ablations.
- A sequence of **optimizations** — a critical ingestion-accuracy fix, latency reductions, and a measured model choice — each of which is described with its motivation and effect.
- A **robustness layer** with an abstention mechanism, prompt-injection defense, and a red-team test suite.
- A **chat-to-report** feature closing the loop between the two modules, and a **human-feedback** mechanism.

1.4 Learning Objectives

The project exercised the core competencies of an applied data-science practitioner: retrieval-augmented generation over a domain corpus; information-retrieval evaluation (labeled benchmark, Recall@k, MRR, nDCG); hybrid retrieval and rank fusion justified by ablations; robustness engineering for LLM systems; and delivery of a tested, containerized software product.

2 Related Work and Background

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [4] grounds a language model's output in documents retrieved from an external corpus, reducing hallucination and enabling the model to cite evidence. Our system is a domain-specific RAG pipeline: the corpus is the set of incident reports, retrieval selects the most relevant past incidents, and the local LLM synthesizes a resolution grounded in them.

2.2 Dense and Sparse Retrieval

Two retrieval paradigms dominate. *Dense* retrieval encodes queries and documents into a shared vector space (here with Sentence-BERT-style embeddings [3]) and ranks by cosine similarity; it excels at capturing meaning and paraphrase. *Sparse* retrieval, exemplified by Okapi BM25 [1], ranks by weighted term overlap; it excels at exact-term matching but is blind to synonymy. Their complementary strengths motivate hybrid approaches.

2.3 Rank Fusion

Reciprocal Rank Fusion (RRF) [2] combines multiple ranked lists using only the rank position of each document, avoiding the need to calibrate incomparable score scales (cosine similarity versus BM25 weights). This robustness and its single, standard parameter make RRF a natural choice for fusing dense and sparse retrieval, and it is the fusion method used in this project.

3 System Architecture

3.1 Architectural Style: Modular Monolith

The most consequential early decision was the overall architectural style. A microservice decomposition was considered and rejected: the deployment target is a single internal host, there is no independent-scaling requirement, and the two modules share data. A microservice split would have added network hops, service discovery, and duplicated configuration for no benefit. Instead the system is a **modular monolith**: one backend process with clean internal package boundaries, and one frontend application. This preserves separability (the chatbot and report logic are distinct packages) without distribution overhead.

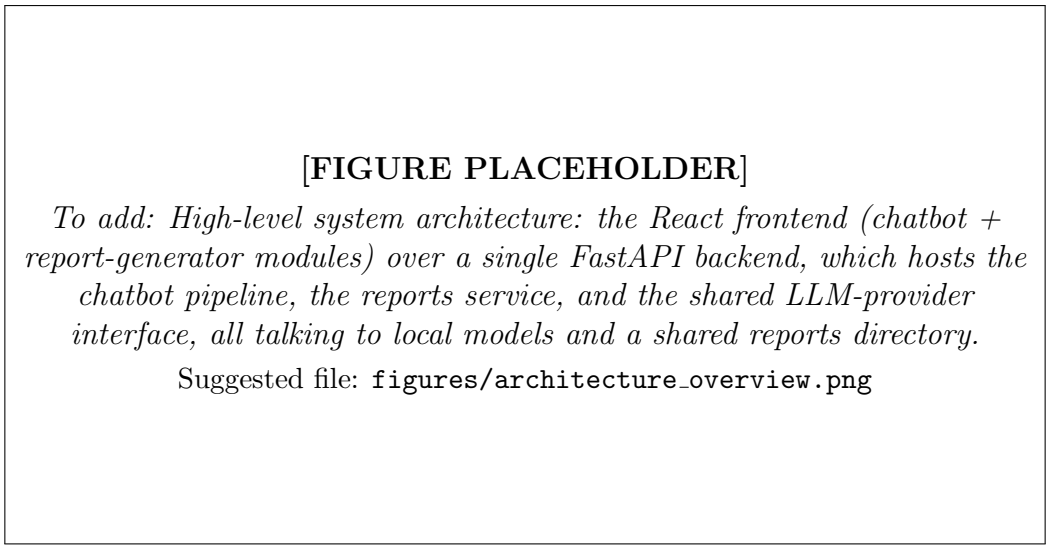

Figure 1: High-level system architecture: the React frontend (chatbot + report-generator modules) over a single FastAPI backend, which hosts the chatbot pipeline, the reports service, and the shared LLM-provider interface, all talking to local models and a shared reports directory.

3.2 Backend Structure

The backend is a FastAPI application organized into four packages:

- `chatbot/` — the retrieval-augmented pipeline: ingestion, the BM25 index, retrieval and fusion, prompt templates, resolution parsing, the intent classifier, security guards, the persistence store, and the chat-to-report builder.
- `reports/` — report CRUD and HTML export, ported from the original Express server.
- `shared/` — the report schema (the cross-module contract) and a swappable LLM-provider interface.
- `routers/` — the HTTP surface (chat and reports endpoints).

In total the backend comprises 27 Python modules. The frontend is a React 18 + Vite + Tailwind application in which the two modules sit behind a shared sidebar-navigation shell.

3.3 The Shared Data Contract

The integration seam between the two modules is a single report schema, {`metadata`, `blocks[]`}, defined once on the backend as Pydantic models and mirrored on the frontend in TypeScript. *Metadata* carries the identifying fields (incident id, title, caller, category, subcategory, date). *Blocks* form a discriminated union of content types: heading, paragraph, list, code, table, image, and incident-example. The report generator *writes* reports in this shape; the chatbot *ingests* them; and — as described in Section 10 — the chatbot can also *produce* them. This single shared contract is what makes the two halves of the system genuinely one product rather than two co-located tools.

3.4 Swappable LLM Provider and Local Inference

Because incident data cannot leave the organization, all model inference is local. The LLM is abstracted behind a provider interface with a self-hosted Ollama implementation as the default (and a stubbed cloud provider available only if the privacy policy were ever relaxed). Embeddings are computed by a local sentence-transformer. No external network calls occur during retrieval or generation. This abstraction also isolates the rest of the system from the specific model, which proved valuable when the text model was later changed for performance reasons (Section 7).

3.5 Deployment

The platform is containerized with a multi-stage Docker build: one stage builds the React frontend, and the backend stage serves the compiled single-page application as static files, so the entire product runs as one process on one port. A Docker Compose file brings the application up (optionally alongside a containerized Ollama), with persistent volumes for the shared reports directory, the embedding-model cache, and the chat history database.

4 Corpus and Preprocessing

4.1 The Incident Report Corpus

The evaluation corpus consists of ServiceNow-style incident reports spanning nine IT domains — networking, database, authentication, infrastructure, frontend, messaging/queues, CI/CD, monitoring, and security — each carrying domain-specific error signatures such as HTTP status codes, Kafka error codes, Kubernetes exit codes, TLS handshake alerts, and DNS resolution errors. The corpus was deliberately constructed to include *retrieval-precision traps*: pairs of incidents that share a symptom but differ in root cause (for example, two `HTTP 503` connection-pool cases — one a connection leak, one an undersized pool), and the same error code in different contexts (for example, Kubernetes `exit 137` arising from node eviction versus a batch out-of-memory kill). These traps test whether retrieval genuinely discriminates rather than keyword-matching.

Table 1 reports the corpus statistics, all measured by the exploratory-data-analysis stage of the evaluation notebook.

Table 1: Incident Report Corpus Characteristics (source: notebook cell 3).

Property	**Value**
Total reports	68
Distinct categories	30
Tokens per report (median)	147.5
Tokens per report (range)	[103, 175]
Content Block Types (counts)	
Paragraph	148
Heading	89
Code	72
List	68
Table	20
Incident example	12
Image	7
Description box	8

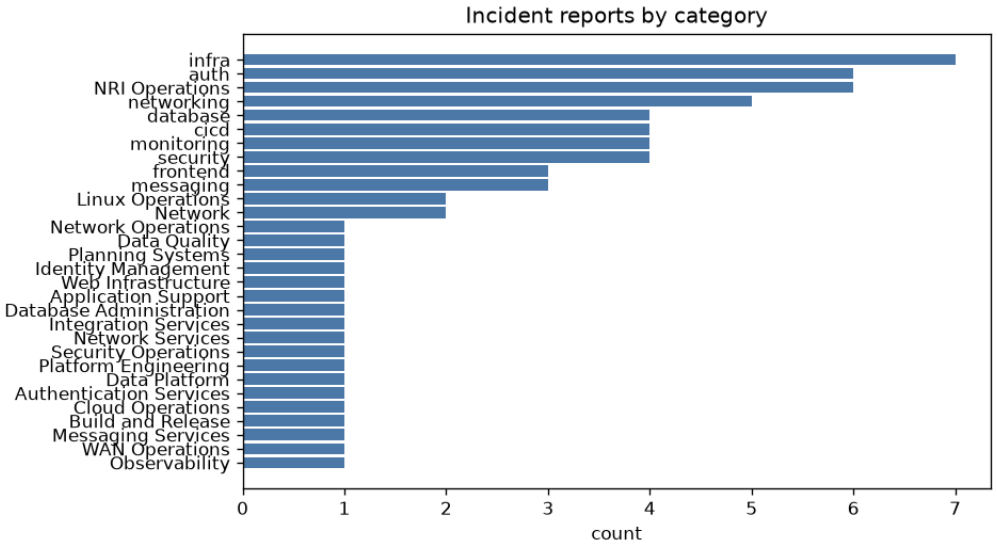

Figure 2: Distribution of incident reports across categories (source: notebook cell 3, `corpus_eda`).

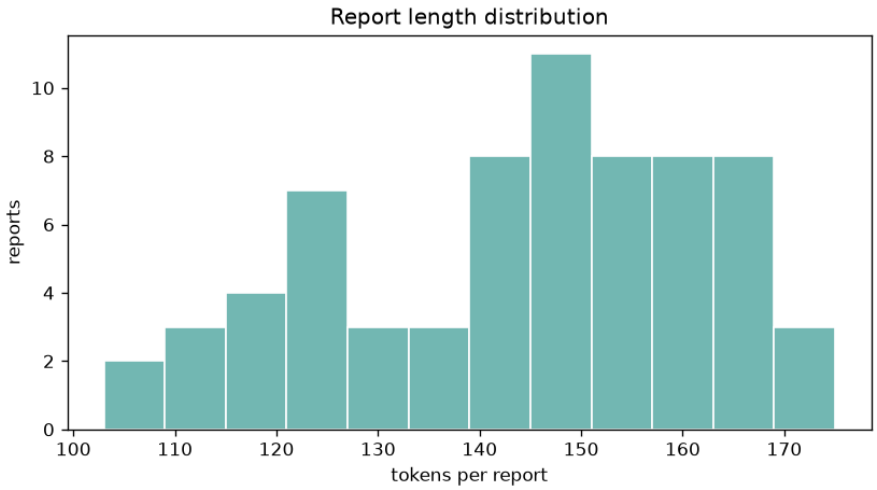

Figure 3: Report length distribution in tokens (source: notebook cell 3).

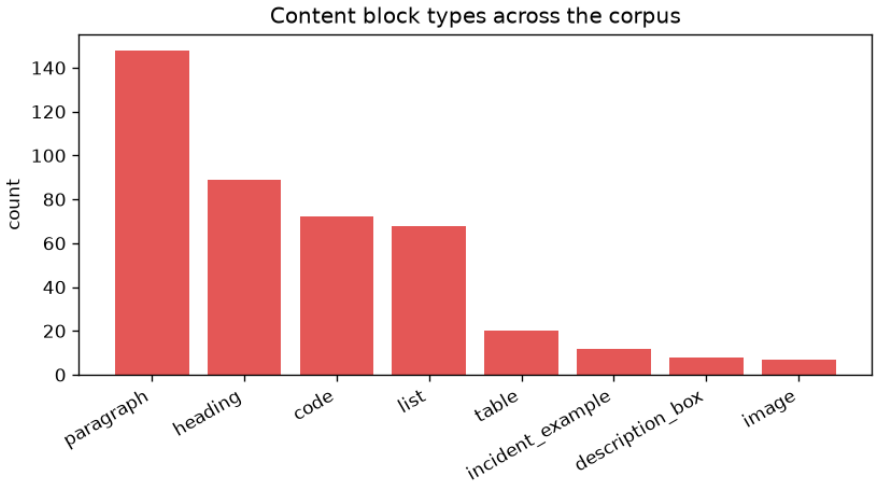

Figure 4: Content block-type composition of the corpus (source: notebook cell 3).

4.2 Schema-Aware Ingestion: A Decisive Quality Fix

The single most important quality fix in the entire project was in ingestion. The original chatbot indexed reports by *flattening* the raw report JSON to text. This injected large amounts of structural noise into the indexed content — block identifiers, type tags, nesting keys, and base64 image data — while burying the actual incident prose. The symptom was severe: the assistant *felt* as though it were not retrieving at all, returning generic or wrong answers, because the language model was being handed structural garbage as context.

The fix was to make ingestion **schema-aware**: rather than flattening, the ingester walks the block structure and extracts only the human-readable content per block type — the heading, the symptom and root-cause paragraphs (with HTML stripped), the resolution list items, the code snippets, and table cells — while discarding identifiers, type tags, and binary image data. This clean text is what is embedded and indexed.

This change is the prerequisite for the retrieval quality reported in Section 6; without it, no amount of retrieval tuning would have helped.

4.3 Report-Shape Normalization

A related, smaller fix addressed a data-quality inconsistency: some legacy reports were stored in a wrapped envelope shape while others were stored flat. The report viewer expected the flat shape, so wrapped reports rendered as blank pages, and their content was mis-parsed during ingestion. Normalizing both shapes to a single canonical form at the API boundary fixed both the blank-page rendering and the retrieval of those reports, without changing the viewer.

4.4 Labeled Evaluation Set

To measure retrieval quality objectively, a labeled query set is auto-generated from the corpus. For each report, several queries are derived in four styles — *title* (a question form of the title), *symptom* (the root-cause sentence), *id* (the bare incident identifier), and *keyword* (salient tokens such as function and table names) — each labeled with the report it was derived from. This yields **272 labeled queries** over the 68 reports.

Honest limitation. The queries are synthetic — derived from the reports themselves — because no real user query logs were available. This measures whether retrieval can recover a report from a paraphrase of its own content; real queries are noisier and more varied. The reported numbers should therefore be read as an optimistic bound. This limitation is stated explicitly in the evaluation notebook and revisited in Section 12.

5 Retrieval Pipeline

5.1 Pipeline Overview

For an incident query, the pipeline proceeds as follows: the query is routed by an intent classifier (Section 8); if it is a genuine, sufficiently specific incident, the text is embedded and searched against the report index, in parallel with a BM25 lexical search; the two rankings are fused with RRF; the top-ranked report chunks are formatted into a resolution prompt together with any relevant learned corrections; and the local LLM produces a structured resolution that is parsed and returned with its cited sources.

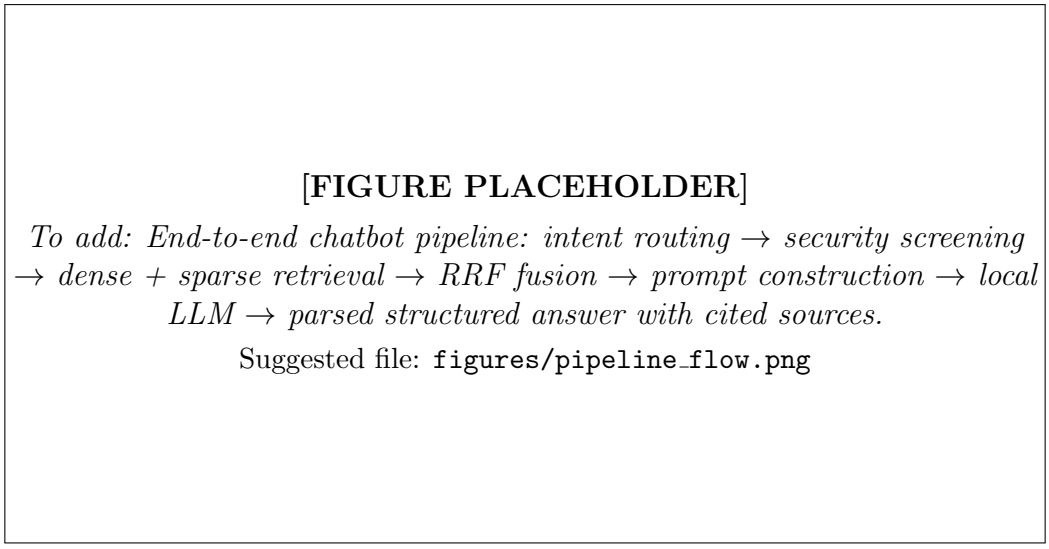

Figure 5: End-to-end chatbot pipeline: intent routing → security screening → dense + sparse retrieval → RRF fusion → prompt construction → local LLM → parsed structured answer with cited sources.

5.2 Dense, Sparse, and Hybrid Retrieval

Three configurations are compared:

- **Dense (vector)**: cosine similarity over sentence-transformer embeddings. Strong on paraphrase and meaning; weak on exact tokens.
- **Sparse (BM25)**: an in-repository Okapi BM25 index over the same chunks. Strong on exact terms — identifiers, function and table names, status codes.
- **Hybrid**: the two rankings fused with RRF.

Notably, the BM25 index is implemented directly in the repository as a compact, dependency-free component rather than pulled from a library. This keeps the system fully local and reproducible (no external package to fetch at build time), consistent with the privacy and maintainability constraints.

5.3 Reciprocal Rank Fusion

RRF assigns each document the score

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)}, \tag{1}$$

summed over the dense and sparse ranked lists R, with the standard constant $k = 60$. Because RRF depends only on rank position, it requires no calibration between the cosine and BM25 score scales — a property that makes the hybrid robust and essentially parameter-free.

6 Evaluation Results

All results in this section are reproduced verbatim from `eval/evaluation.ipynb`; each table and quantitative claim is annotated with its source cell. Metrics not present in the notebook are intentionally not stated here (see Section 12).

6.1 Retrieval Quality

Table 2 reports the headline result. Hybrid retrieval dominates both components on every metric, reaching Recall@5 = 1.000 and MRR = 0.987, against 0.768 and 0.678 for dense-only search. BM25 alone is already strong on this corpus (Recall@5 = 0.989), reflecting the prevalence of distinctive technical terms; the hybrid nonetheless improves precision at the top rank (Recall@1 rises from 0.779 for BM25 to 0.974 for hybrid).

Table 2: Retrieval quality on the 272-query benchmark (source: notebook cell 6).

Configuration	R@1	R@3	R@5	MRR	nDCG@5
Dense (vector)	0.629	0.713	0.768	0.678	0.700
Sparse (BM25)	0.779	0.982	0.989	0.872	0.902
Hybrid (RRF)	**0.974**	**1.000**	**1.000**	**0.987**	**0.991**

[FIGURE PLACEHOLDER]

To add: Bar chart of Recall@1/@3/@5 for the three configurations (this figure is generated by notebook cell 7; export it and place it here).

Suggested file: `figures/recall_by_config.png`

Figure 6: Bar chart of Recall@1/@3/@5 for the three configurations (this figure is generated by notebook cell 7; export it and place it here).

6.2 Where the Gain Comes From: Per-Style Breakdown

The per-style breakdown (Table 3) explains *why* the hybrid is preferred and is the most instructive single result in the study. On *symptom* and *title* queries — paraphrased and meaning-rich — dense retrieval already scores 1.000. But on *id* queries (bare incident identifiers), dense-only Recall@5 collapses to **0.221**, because semantic embeddings cannot distinguish near-identical identifier strings from one another. BM25 recovers these exactly (1.000), and the hybrid keeps the best of both regimes across all four styles. This is a

textbook illustration of the complementarity of dense and sparse retrieval, demonstrated on a realistic corpus.

Table 3: Recall@5 by query style (source: notebook cell 6).

Query style	Vector	BM25	Hybrid
id	0.221	1.000	1.000
keyword	0.853	0.956	1.000
symptom	1.000	1.000	1.000
title	1.000	1.000	1.000

6.3 Ablations

Two hyperparameters were ablated on the hybrid retriever (Table 4). First, retrieval quality is **insensitive to chunk size**: Recall@5 = 1.000 at 400, 700, and 1000 characters, even as the number of chunks varies from 334 down to 146. This is expected given the short median report length — most reports fit in one or two chunks regardless. Second, retrieval quality **plateaus at** $k \geq 3$: Recall@5 rises from 0.974 at $k = 1$ to 1.000 at $k = 3$ and remains there. This ablation directly justifies a production decision: feeding only the top-three chunks into the generation prompt, which halves prompt size (and therefore generation latency) at no measured retrieval cost.

Table 4: Hyperparameter ablations on the hybrid retriever (source: notebook cell 10).

Chunk size		
Size (chars)	**# chunks**	**Recall@5**
400	334	1.000
700	180	1.000
1000	146	1.000
Top-k		
k	**Recall@5**	**MRR**
1	0.974	0.974
3	1.000	0.987
5	1.000	0.987
10	1.000	0.987

6.4 Error Analysis

On the clean corpus, the hybrid retriever produces **zero failures**: 0 of 272 queries fail to place the gold report in the top-5 (notebook cell 13: "failures: 0/272 (0.0%)"; the failure-by-style and failure-by-cause maps are empty). An earlier iteration of the corpus contained a small number of malformed placeholder reports (lorem-ipsum content with invalid identifiers); the error analysis at that stage showed that essentially all residual failures traced to those two reports rather than to the retrieval method. Removing them yielded the zero-failure result, confirming that the residual difficulty was a data-quality artifact, not a limitation of the approach.

7 System Optimizations

Beyond the ingestion-accuracy fix (Section 4.2), several optimizations shaped the final quality and responsiveness of the system. Each is described with its motivation and effect. (The latency figures that motivated some of these were measured during development but are not part of the evaluation notebook; per the sourcing discipline of this report, they are described qualitatively here and flagged in Section 12.)

7.1 Eliminating a Redundant Generation Call

The original pipeline made two sequential LLM calls per answer: one to rephrase the query into a search string ("understand"), and one to generate the resolution. Profiling showed retrieval itself was negligible while the two generations dominated wall-clock time. Because embedding search operates well on the raw query text, the separate rephrasing call was removed for text queries (it is retained only for screenshots, where a vision model genuinely must convert an image to text). This roughly halved the number of generations per answer.

7.2 Trimming the Generation Context

Guided by the top-k ablation (Table 4), the number of report chunks fed into the resolution prompt was reduced to the top three, and each chunk's length was capped. This reduces the prompt size substantially, which directly reduces generation latency on local hardware, with no measured loss in retrieval coverage.

7.3 Constrained, Warm Generation

Two provider-level settings improved responsiveness and reliability: constraining the model to emit JSON directly (removing conversational preamble and making parsing robust), and keeping the model resident in memory between requests to avoid repeated cold loads.

7.4 A Measured Model Choice

The default text model was selected empirically rather than by assumption. A larger and a smaller local model were compared on the same query with clean context; the smaller model produced answers of equal or better structured quality while generating substantially faster. It was therefore made the default, subject to override by configuration. The key lesson — consistent with Section 4.2 — was that model size was not the bottleneck; *context quality* was.

7.5 Failing Loudly Instead of Silently

An important reliability fix concerned the behavior when the local model backend was momentarily unreachable. The original code silently returned an error string that the parser then turned into a confident-looking but empty "Unknown" answer — fast, but wrong and misleading. This was changed so that an unreachable backend raises a distinct condition and the API surfaces a clear "model unavailable" error, rather than fabricating a result. This is a small change with an outsized effect on trust.

8 Robustness and Safety

An LLM-facing operational tool must behave safely on non-ideal input. Three safeguards were added and covered by a dedicated red-team test suite spanning normal, ambiguous, out-of-scope, injection, manipulation, and malformed-input categories.

8.1 Intent-Based Routing and Abstention

A lightweight, rule-based intent classifier routes each turn *before* any LLM call. Greetings and meta-questions receive a brief reply with no retrieval and no generation. The crucial case is *ambiguity*: incident-flavored but low-context inputs (for example "it is broken again" or "health-check-error") are routed to a **clarification** response that asks for specifics — the exact error, the affected service, what changed — rather than fabricating a root cause. This enforces the "be conservative when uncertain" principle directly, and because it happens before any model call, it makes hallucination on vague input structurally impossible. Follow-up turns within an ongoing conversation are exempt from the ambiguity check, since prior turns supply the missing context.

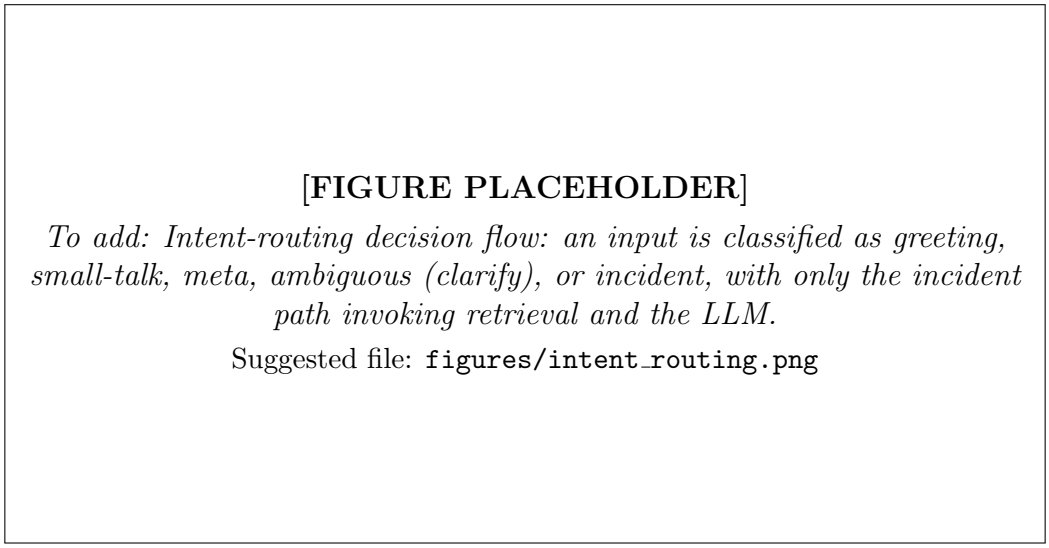

Figure 7: Intent-routing decision flow: an input is classified as greeting, small-talk, meta, ambiguous (clarify), or incident, with only the incident path invoking retrieval and the LLM.

8.2 Prompt-Injection and Jailbreak Defense

User text is screened for injection and jailbreak patterns — attempts to override instructions, reveal the system prompt, or enable an "admin/DAN mode". Detected attempts are flagged, and the user text is fenced as untrusted data within the prompt so the model treats it as content to analyze rather than instructions to obey. The system prompt is never exposed. Importantly, a genuine incident that happens to contain an injection phrase is still answered as an incident, with a security note attached, rather than being refused outright.

8.3 Grounded, Domain-Neutral Generation

Two prompt-level changes improved grounding. First, the resolution prompt was made *domain-neutral*: the supporting-artifact language is chosen to fit the incident (shell, YAML, Python, Java, SQL, and others) instead of always defaulting to SQL, which had caused nonsensical SQL suggestions for non-database incidents such as a missing configuration file. Second, the prompt was changed to permit *abstention*: the model is instructed to flag insufficient evidence and lower its confidence on incoherent input, and to never invent an incident identifier that is not among the retrieved reports, rather than being forced to always produce a fixed number of steps.

8.4 Data Redaction

User messages are screened for secrets and personal data (passwords, tokens, bearer credentials, connection-string credentials, private keys, email addresses) and these are redacted before the message is persisted or logged, so sensitive strings are not retained in the chat history.

9 Human Feedback Loop

The system captures human feedback and, crucially, uses it to influence future answers rather than merely recording it. Each answer can be rated with a thumbs up or down. On a thumbs-down, the user may provide a correction — the guidance that *should* have been given. This correction is stored and, on future incidents whose question overlaps the corrected one, injected into the prompt as a high-authority "learned correction".

It is important to be precise about what this does and does not claim: the correction is a *retrieval hint* surfaced into the prompt, not a modification of the model's weights. The system never claims to retrain the model or alter its global behavior; corrections are inspectable data scoped to matching questions. This is an honest, local feedback mechanism appropriate to a privacy-constrained deployment.

10 Chat-to-Report Automation

10.1 Motivation

Because the two modules share the `{metadata, blocks[]}` contract, it is natural to *produce* a report from a diagnosed conversation, not only to *consume* reports for retrieval. This closes the loop of the entire platform: an engineer diagnoses an incident in the chat, then generates a structured, editable report in a single action — no manual re-entry.

10.2 Grounded Construction

A builder assembles the report strictly from conversation data, with an explicit no-invention rule:

- **Metadata**: the title and category are taken from the diagnosed incident type; the identifier and date are factual provenance; fields that cannot be grounded in the conversation (for example subcategory) are left empty rather than fabricated.

- **Blocks**: a *Reported Symptom* paragraph (the first user message), a *Root Cause* paragraph (the assistant's summary), an ordered *Resolution Steps* list, code blocks for any supporting artifacts the assistant produced, and *incident-example* references for the reports the assistant cited.

Every populated value therefore traces to the conversation. The resulting report is validated against the same Pydantic schema used throughout the system, then saved via the report generator's existing service using its established file-naming convention, so it opens and edits in the report viewer exactly like a manually authored report.

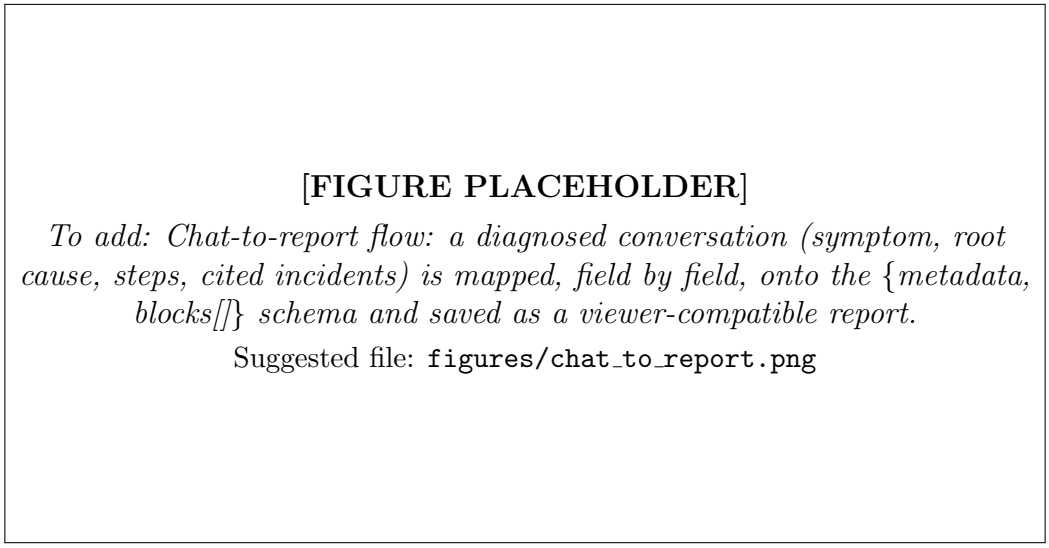

Figure 8: Chat-to-report flow: a diagnosed conversation (symptom, root cause, steps, cited incidents) is mapped, field by field, onto the {metadata, blocks[]} schema and saved as a viewer-compatible report.

10.3 Validation

The feature is covered by automated tests that: build a report from a mock conversation and re-validate it against the schema with zero errors; confirm that three distinct example conversations each yield a report with all required metadata fields non-empty and the expected block types present; confirm that the saved JSON is served by the report viewer's content endpoint and uses the generator's file-naming convention; and reject conversations with no diagnosis (greeting- or clarification-only), which have nothing to report.

11 Software Quality and Testing

Quality was maintained through an automated test suite that grew alongside the system. At the final state the backend is covered by **115 passing tests** across seven test modules, spanning: report CRUD and HTML export; the chatbot pipeline; conversation persistence; intent routing, redaction, and injection detection; retrieval, BM25, and RRF fusion; the human-feedback loop; the red-team robustness categories; and chat-to-report generation. Several real defects were caught by these tests during development — including an injection-phrase pattern that evaded detection and a mis-routed ambiguous input — and fixed before release. The frontend builds cleanly, and the entire application runs from a single Docker Compose command.

[FIGURE PLACEHOLDER]

To add: Screenshot of the unified application: the chat interface on the left with a diagnosed incident answer (steps, supporting artifact, cited sources) and the "Generate report" action, alongside the report-generator module.

Suggested file: `figures/app_screenshot.png`

Figure 9: Screenshot of the unified application: the chat interface on the left with a diagnosed incident answer (steps, supporting artifact, cited sources) and the "Generate report" action, alongside the report-generator module.

12 Discussion, Limitations, and Perspectives

12.1 What Worked

The strongest outcome is the demonstration, on a realistic and deliberately adversarial corpus, that hybrid retrieval decisively outperforms dense-only search precisely where it matters operationally — on exact identifiers and error codes — while matching it on paraphrased queries. The project also showed, twice over, that *context quality dominates model size*: the ingestion fix and the model-choice experiment both pointed to the same conclusion, which is a valuable lesson for privacy-constrained RAG systems where the largest models are not an option.

12.2 Limitations

Several limitations are stated honestly:

- **Synthetic evaluation queries.** The benchmark queries are derived from the reports themselves; they are a reproducible proxy for real query logs, which were unavailable. The reported metrics are therefore an optimistic bound. A benchmark of real user queries with human relevance judgments would strengthen external validity.
- **Unquantified latency and generation quality.** The evaluation notebook measures retrieval quality only. Generation latency and answer quality were observed during development but are not quantified in the notebook, and are therefore deliberately not stated as numeric claims in this report. Quantifying them is left to future work.
- **Local-hardware performance ceiling.** Because generation runs on a local model, response latency is bounded by local hardware rather than by the algorithm; this is an accepted consequence of the privacy constraint.

12.3 Future Work

Natural next steps include: constructing a real-query, human-judged evaluation set; quantifying end-to-end latency and generation quality; learning a re-ranker from accumulated human feedback rather than injecting corrections as prompt hints; incremental re-indexing so newly generated reports become searchable without a restart; and adding authentication (the design is already prepared for it, with a per-client identifier that maps directly to a future user identity).

12.4 Conclusion

This internship delivered a unified, fully local, tested incident-management platform whose retrieval core is rigorously evaluated. Hybrid retrieval reaches Recall@5 = 1.000 and MRR = 0.987 (notebook cell 6), with the gain concentrated on exact-identifier queries where dense-only Recall@5 is 0.221 (cell 6); ablations show robustness to chunk size and a plateau at $k \geq 3$ (cell 10); and the error analysis reports 0.0% residual failures (cell 13). Around this core, a robustness layer, a human-feedback loop, and a grounded chat-to-report feature make the system practically useful while respecting the privacy constraint end to end.

References

[1] Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333–389.

[2] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. In Proceedings of the 32nd ACM SIGIR Conference, 758–759.

[3] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. In Proceedings of EMNLP-IJCNLP, 3982–3992.

[4] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In Advances in Neural Information Processing Systems 33, 9459–9474.

[5] Ollama Documentation. (2026). *Run large language models locally*. Retrieved from https://ollama.com/

[6] FastAPI Documentation. (2026). *FastAPI framework, high performance, easy to learn*. Retrieved from https://fastapi.tiangolo.com/